



INDIA.RUNS

Build what next India runs on

Team Name : RushOne

Team Leader Name : Sagar

Problem Statement : Intelligent Candidate Discovery

Solution Overview

- **A multi-signal Python ranking engine that scores 100,000 candidates against a job description in under 90 seconds on CPU — no GPU, no external APIs.**
- **Combines 5 scoring dimensions: skill match (35%), experience fit (20%), title relevance (15%), behavioral signals (20%), and location preference (10%).**
- **Unlike keyword-only matchers, our system understands context: a Marketing Manager with 10 AI keywords scores 0.00, while an ML Engineer with real retrieval experience scores 0.75+.**
- **Built-in honeypot detection automatically zeroes out impossible profiles (e.g. 'expert' in 5 skills with 0 months used), protecting submission quality.**

JD Understanding & Candidate Evaluation

- **Key JD requirements extracted: Python, NLP, embeddings, sentence-transformers, FAISS/Pinecone/Qdrant, BM25, dense/hybrid retrieval, NDCG/MRR evaluation, 5–9 years experience, Pune/Noida location.**
- **Most important candidate signals: current job title (ML/NLP/AI engineer vs. non-technical), career history descriptions, skill proficiency level, and whether experience is at product vs. services companies.**
- **Beyond keyword matching: we scan full-text career descriptions for implicit signals (e.g. 'shipped recommendation system' = ranking experience even without the word 'ranking').**
- **Behavioral signals used: open-to-work flag, last active date, recruiter response rate, notice period, GitHub activity score, and interview completion rate — inactive candidates rank lower regardless of skills.**

Ranking Methodology

- **Step 1 — Skill Score (35%):** Match required + nice-to-have skills across skills list, headline, summary, and career descriptions. Expert/advanced proficiency adds bonus.
- **Step 2 — Experience Score (20%):** Ideal 6-8 yrs = 1.0; acceptable 5-9 yrs = 0.75-0.85; outside range = penalty. Product company history adds bonus.
- **Step 3 — Title Score (15%):** ML/NLP/AI Engineer = 1.0; Marketing/HR/Content/Sales = 0.05 (strong negative); Senior/Lead neutral = 0.6.
- **Step 4 — Behavioral Score (20%):** 12 sub-signals — open-to-work (15pts), activity recency (20pts), response rate (15pts), notice period (10pts), GitHub (10pts), and 7 more.
- **Step 5 — Location Score (10%):** Pune/Noida/Delhi NCR/Hyderabad/Bangalore = 1.0; other India = 0.5; outside India = 0.1.
- **Final = weighted sum x honeypot_penalty (0.0–1.0).** Impossible profiles score 0.00 and are excluded.

Explainability & Data Validation

- **Every candidate output includes a factual 1–2 sentence reasoning built from actual profile fields: title, years of experience, location, specific skills found, response rate, notice period, and GitHub score.**
- **No hallucination: the reasoning function only references data fields that exist in the candidate object. If a field is missing, it is silently skipped — never invented.**
- **Honeypot/fraud detection via honeypot_penalty(): profiles with 5+ expert-level skills at 0 months usage score 0.00 and are excluded from the top 100. Keyword stuffers (many expert skills, non-technical headline) receive a 0.2x multiplier.**
- **Low-quality profiles handled gracefully: missing fields default to neutral values, never causing crashes or inflated scores. Incomplete profiles naturally rank lower due to lower behavioral scores.**

End-to-End Workflow

- **Step 1 — INPUT:** Read `candidates.jsonl.gz` (100K candidates, ~52MB) via Python `gzip+json` streaming.
- **Step 2 — SCORE:** Compute 5 sub-scores per candidate. Pure Python arithmetic on JSON fields — no ML inference, no API calls.
- **Step 3 — PENALIZE:** Apply `honeypot_penalty()` multiplier. Impossible/fraudulent profiles → score = 0.0.
- **Step 4 — RANK:** Sort all 100K candidates by final score descending. Select top 100.
- **Step 5 — REASON:** Call `build_reasoning()` per top-100 candidate — generates factual 1-2 sentence justification from actual profile data.
- **Step 6 — OUTPUT:** Write `submission.csv` with columns: `candidate_id`, `rank`, `score`, `reasoning`.
- **Runtime:** ~60-90 seconds CPU. Memory: ~2GB peak. Meets all challenge compute constraints.

System Architecture

- **rank.py** — **Single-file ranker. Five scoring modules: skill_score(), experience_score(), title_score(), behavioral_score(), location_score().**
- **honeypot_penalty()** — **Fraud detection: checks for impossible skill durations, keyword stuffing, suspicious tenure patterns.**
- **build_reasoning()** — **Explainability: reads actual profile fields and generates honest, factual 1-2 sentence justification per candidate.**
- **app.py** — **Streamlit sandbox. Upload any JSONL subset, get ranked CSV output instantly. Hosted free on Streamlit Cloud.**
- **Data flow: candidates.jsonl.gz → load_candidates() → rank_candidates() → write_submission() → submission.csv**

Results & Performance

- **Test result: Strong ML Engineer (Python, sentence-transformers, FAISS, NDCG, 7 yrs, Pune, open-to-work) → Final score: 0.7538.**
- **Test result: Marketing Manager with 10 AI keywords stuffed in skills, 0 months used, inactive 8 months → Final score: 0.0000 (honeypot detected).**
- **Runtime benchmark: ~60–90 seconds for 100,000 candidates on a standard laptop CPU — well within the 5-minute compute constraint.**
- **Memory usage: ~2GB peak RAM — within the 16GB constraint. No GPU required. No network calls during ranking.**
- **Submission format: 100 rows, columns [candidate_id, rank, score, reasoning]. Validated with the official validate_submission.py before upload.**

Technologies Used

- **Python 3.10+ (stdlib only for core ranker) — chosen for simplicity, speed, and zero-dependency deployment on any CPU machine.**
- **gzip + json (stdlib) — streams 52MB compressed candidate file without loading everything into RAM at once.**
- **Streamlit — for the required hosted sandbox demo. Free deployment on Streamlit Cloud in under 5 minutes.**
- **No sentence-transformers or ML libraries in the core ranker — deliberate choice to meet the CPU-only, 5-minute, 16GB constraint without risk.**
- **GitHub — version control and submission asset. .gitignore excludes the 52MB dataset file (standard ML practice).**
- **All tools declared: Claude (Anthropic) assisted with code structure and documentation.**

Submission Assets

- **GitHub Repository:** `github.com/[your-username]/redrob-ranker` — contains `rank.py`, `app.py`, `README.md`, `requirements.txt`, `.gitignore`
- **Streamlit Sandbox Demo:** (<https://rankingcandidate-rn3cstdvjfjtcpbys3vjms.streamlit.app/>)—upload any JSONL subset, get ranked CSV output instantly.
- **Submission CSV:** `submission.csv` — top 100 candidates, ranked best-fit first, with factual 1-2 sentence reasoning per row.
- **Run command:** `python rank.py --candidates ./candidates.jsonl.gz --out ./submission.csv`
- **AI Tools Declaration:** Claude (Anthropic) used for code structure, scoring logic, and README documentation.

redrob i iZS

INDIA.RUNS

Build what next India runs on

THANK YOU